SAE Base De Données SQL

Image provenant de istockphoto.com

INTRODUCTION 2
PARTIE 1 2
1.1 SCRIPT SQL de création des tables..... 2
PARTIE 2 4
2.1 Illustrations comparatives Cours/AGL association fonctionnelle 4
2.2 Illustrations comparatives Cours/AGL association maillée 5
2.3 Modèle physique de données réalisé avec l'AGL 7
2.4 Script généré par l'AGL 7
2.5 Différences entre script manuel et automatique 9
PARTIE 3 11
3.1 Script de peuplement de la base de données 11
3.2 Description des différentes étapes du script de peuplement..... 15
CONCLUSION 17

INTRODUCTION

Le monde connaît de plus en plus de catastrophes climatiques comme les inondations, la sécheresse, les tempêtes etc. Il est donc crucial de pouvoir analyser d'avoir des outils permettant d'analyser et suivre ces phénomènes.
Ainsi, grâce à cette SAÉ, nous initie à la création de base de données solide dans le cas des fréquences des catastrophes climatiques.

Ainsi nous ferons dans un premier temps un script création de tables vide, puis dans un second temps nous nous pencherons sur la découverte d'une nouvelle modélisation qui générera un script qu'on comparera à notre premier script et enfin nous insérerons des données dans les tables créées.

Partie 1

Script manuel de création de base de données

```
CREATE TABLE region (
     region_code INTEGER PRIMARY KEY,
     name VARCHAR NOT NULL
);

CREATE TABLE sub_region (
     sub_region_code INTEGER PRIMARY KEY,
     name VARCHAR NOT NULL,
     region_code INTEGER NOT NULL REFERENCES
     region(region_code) ON DELETE CASCADE
);

CREATE TABLE country (
     country_code INTEGER PRIMARY KEY,
     name VARCHAR NOT NULL,
     iso2 VARCHAR(2),
     iso3 VARCHAR(3) NOT NULL,
     sub_region_code INTEGER NOT NULL REFERENCES
     sub_region(sub_region_code) ON DELETE CASCADE
);
```

```sql
CREATE TABLE disaster (
     disaster_code INTEGER PRIMARY KEY,
     disaster VARCHAR NOT NULL
);

CREATE TABLE climate_disaster (
     country_code INTEGER NOT NULL REFERENCES
     country(country_code) ON DELETE CASCADE,
     disaster_code INTEGER NOT NULL REFERENCES
     disaster(disaster_code) ON DELETE CASCADE,
     year INTEGER NOT NULL,
     number INTEGER NOT NULL,
     PRIMARY KEY (country_code, disaster_code, year)
);
```

Explication du code :

Ce code de création de base de données permet d'avoir les tables "région", "sub_region", "country", "disaster" et "climate_disaster" manuellement avec POSTGRESQL, en utilisant le schéma relationnel du sujet page 3.

- `CREATE TABLE` : permet de créer une table
- `INTEGER, CHAR` : permet de définir le type de valeur dans la colonne
- `PRIMARY KEY` : permet de définir quel élément est la clé
- `NOT NULL` : permet de dire qu'on doit impérativement entrer une valeur pour cette colonne
- `REFERENCES ...` : permet de dire que l'élément est une clé étrangère vers une autre table
- `ON DELETE CASCADE` : permet de supprimer une valeur dans la table actuelle quand un n-uplet qui contient la clé primaire référencée est supprimée dans la table maître. (*cours SQL Laurent Audibert*)

Le sujet ne précise pas d'ajouter certaines contraintes mais ici cela semblerait intéressant d'ajouter `ON DELETE CASCADE` car elle simplifie la gestion des données. Par exemple, si une région est supprimée alors les sous-régions doivent être supprimées pour éviter les incohérences.

TRANSITION:

Nous venons de créer un script qui permet de créer des tables à partir d'un schéma relationnel donné.
Nous essayerons de développer une autre manière de voir les choses à l'aide d'une nouvelle modélisation.

Nous tenterons de comparer les différentes associations entre les maillées et les fonctionnelles et nous finirons par en déduire un nouveau script SQL qu'on comparera au premier.

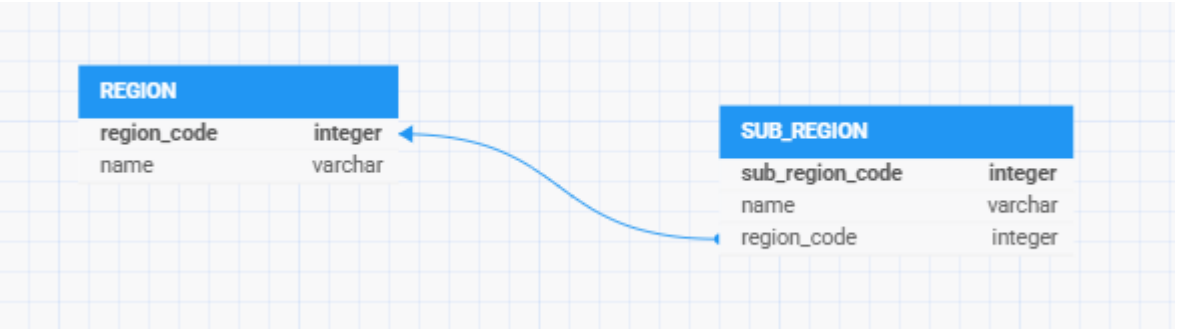
Partie 2

Modélisation et script de création "avec AGL"

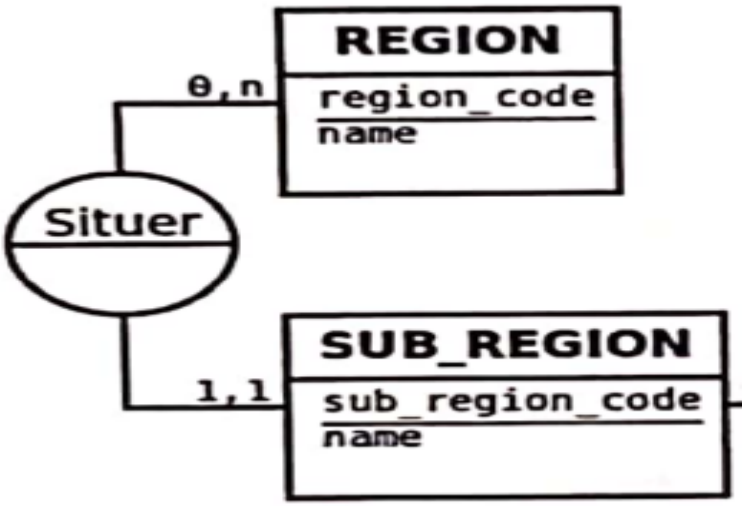
L'Atelier de Génie Logiciel choisi est **DB DESIGNER**.

1. Association fonctionnelle :

a) Version DB DESIGNER

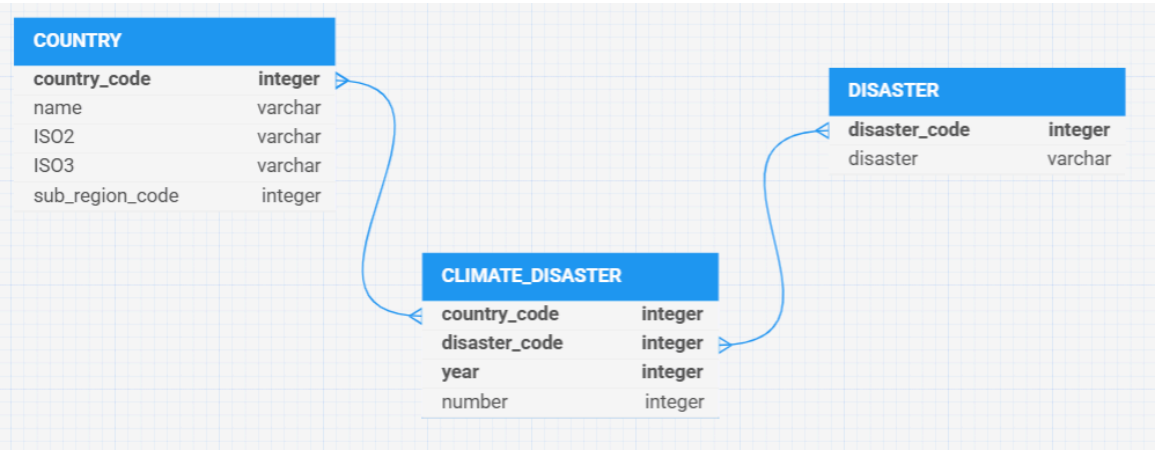


b) Version cours

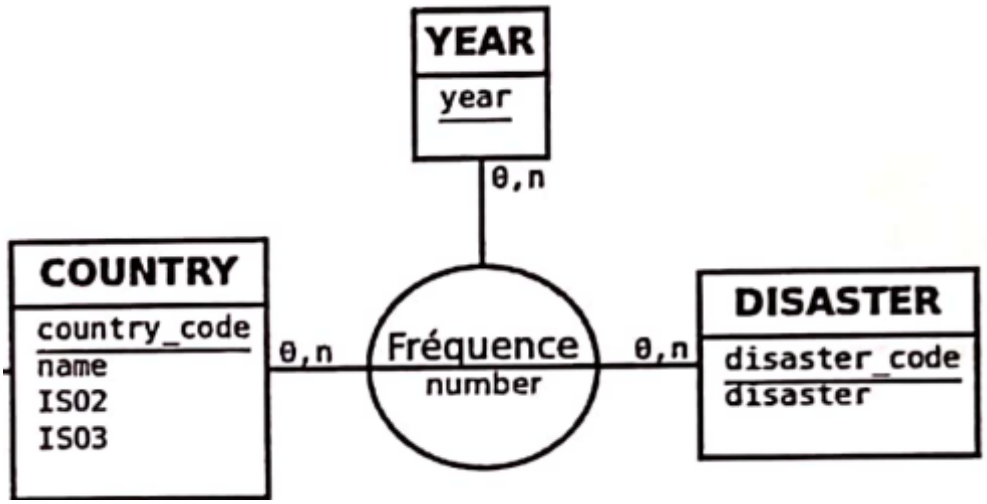


2. Association maillée :

a) Version DB DESIGNER



b) Version cours

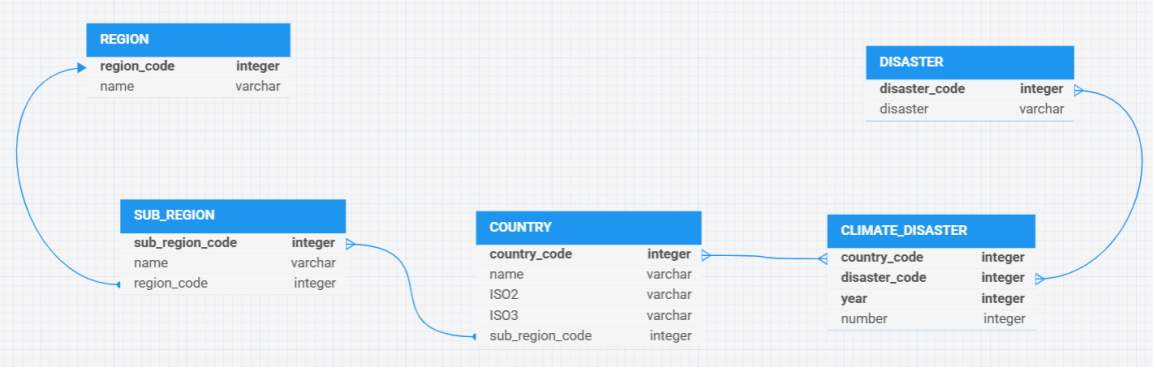


Pour lire le modèle de l'AGL par rapport au modèle vu en cours on doit d'abord identifier les entités principales par exemple pour l'association fonctionnelle c'est "REGION" et "SUB_REGION" qui, au lieu d'être des rectangles sur le modèle vu en classe, ce sera des tables dans le modèle de l'AGL.
Dans ces tables, on trouve des informations, un peu comme le modèle du cours où on y trouve principalement la clé primaire (en gras) et d'autres éléments qui complètent ces tables qui donneront les colonnes.
Cependant, il y a aussi les clés étrangères dans ces tables qui permettent de lier les tables entre elles.

Ensuite, on identifie les associations entre les tables qui sont représentées par des flèches au lieu d'être des losanges dans le modèle du cours.
Ces associations sont couplées aux clés étrangères qui permettent de comprendre facilement comment les tables sont reliées entre elles.
Les flèches partent de la clé étrangère de la table du référent vers la table référencée.

Mais, il faut faire attention à la cardinalité qui n'est pas représentée par des 0 ou 1 ou n (même si on peut, dans certains cas, les représentés par des flèches avec le modèle du cours) mais par des différents types bouts de flèches.
Les cardinalités "plusieurs vers plusieurs" sont représentées par un triangle vers un triangle. (Association maillée)
Les cardinalités "plusieurs vers un" commencent d'un point vers un triangle et les cardinalités "un vers un" forment une flèche. (Association fonctionnelle)

3. Modèle physique de données réalisé avec l'AGL



Ce modèle est la combinaison de l'association fonctionnelle et maillée. Les différences et la lecture de ce modèle ont déjà été traitées.

4. Script généré par l'AGL

```
CREATE TABLE IF NOT EXISTS "REGION" (
     "region_code" serial NOT NULL UNIQUE,
     "name" varchar(255) NOT NULL,
     PRIMARY KEY ("region_code")
);

CREATE TABLE IF NOT EXISTS "SUB_REGION" (

"sub_region_code" serial NOT NULL UNIQUE,
     "name" varchar(255) NOT NULL,
     "region_code" bigint NOT NULL,
      PRIMARY KEY ("sub_region_code")
);

CREATE TABLE IF NOT EXISTS "COUNTRY" (

"country_code" serial NOT NULL UNIQUE,
     "name" varchar(255) NOT NULL,
     "ISO2" varchar(2) NOT NULL,
     "ISO3" varchar(3) NOT NULL,
     "sub_region_code" bigint NOT NULL,
     PRIMARY KEY ("country_code")
);
```

```sql
CREATE TABLE IF NOT EXISTS "CLIMATE_DISASTER" (
     "country_code" bigint NOT NULL,
     "disaster_code" bigint NOT NULL,
     "year" bigint NOT NULL,
     "number" bigint NOT NULL,
     PRIMARY KEY ("country_code", "disaster_code",
"year")
);

CREATE TABLE IF NOT EXISTS "DISASTER" (
     "disaster_code" serial NOT NULL UNIQUE,
     "disaster" varchar(255) NOT NULL,
     PRIMARY KEY ("disaster_code")
);



ALTER TABLE "SUB_REGION" ADD CONSTRAINT "SUB_REGION_fk2"
FOREIGN KEY ("region_code") REFERENCES
"REGION"("region_code");

ALTER TABLE "COUNTRY" ADD CONSTRAINT "COUNTRY_fk4"
FOREIGN KEY ("sub_region_code") REFERENCES
"SUB_REGION"("sub_region_code");

ALTER TABLE "CLIMATE_DISASTER" ADD CONSTRAINT
"CLIMATE_DISASTER_fk0" FOREIGN KEY ("country_code")
REFERENCES "COUNTRY"("country_code");

ALTER TABLE "CLIMATE_DISASTER" ADD CONSTRAINT
"CLIMATE_DISASTER_fk1" FOREIGN KEY ("disaster_code")
REFERENCES "DISASTER"("disaster_code");
```

5. <u>Différences entre les scripts produits manuellement et automatiquement</u>

Création de tables avec le 1er script

```
ordidellm=> \d
                List of relations
 Schema |       Name       | Type  |   Owner
--------+------------------+-------+-----------
 public | climate_disaster | table | ordidellm
 public | country          | table | ordidellm
 public | disaster         | table | ordidellm
 public | region           | table | ordidellm
 public | sub_region       | table | ordidellm
(5 rows)
```

Création de tables avec le 2ème script

```
ordidellm=> \d
                        List of relations
 Schema |              Name              |   Type   |   Owner
--------+--------------------------------+----------+-----------
 public | CLIMATE_DISASTER               | table    | ordidellm
 public | COUNTRY                        | table    | ordidellm
 public | COUNTRY_country_code_seq       | sequence | ordidellm
 public | DISASTER                       | table    | ordidellm
 public | DISASTER_disaster_code_seq     | sequence | ordidellm
 public | REGION                         | table    | ordidellm
 public | REGION_region_code_seq         | sequence | ordidellm
 public | SUB_REGION                     | table    | ordidellm
 public | SUB_REGION_sub_region_code_seq | sequence | ordidellm
(9 rows)
```

<u>Différences :</u>

On peut citer plusieurs différences entre le premier et le deuxième script.

Au niveau des contraintes, les contraintes sur les clés étrangères sont définies dans la définition des colonnes alors que le deuxième script défini cette contrainte après dans des `ALTER TABLE`. Le deuxième code donne un nom explicite des contraintes contrairement au premier.

Il y a des mots-clés qui sont utilisés dans l'un mais pas dans l'autre comme `ON DELETE CASCADE` dans le premier qui n'est pas dans le deuxième et `IF NOT EXISTS`, `UNIQUE`, `NOT NULL` qui n'est pas le premier.

Les types de données sont aussi différents dans le premier on utilise des

`INTEGER`, `VARCHAR` et des `CHAR` sans spécifier la taille à part pour les ISO et dans le deuxième on utilise plutôt des `INTEGER`, `VARCHAR` en spécifiant les tailles maximales et `bigint` (qui a une plage de valeurs plus grandes que int) pour les clés étrangères.

Les clés primaires ne sont pas définies de la même manière. Le premier définit les clés primaires lors de la création de la colonne et l'autre les définit dans une nouvelle ligne. De plus, le deuxième utilise `serial` pour auto-incrémenter les colonnes, au niveau des clés primaires d'où les séquences créés dans la capture d'écran 2.

Au niveau de la portabilité, les différences se trouvent au niveau du type de données et la syntaxe où le premier script utilise une syntaxe plus large et peut être utilisé dans d'autres SGBD alors que le premier utilise des noms à doubles guillemets et `serial` qui sont propre à POSTGRESQL.

Les principaux avantages du premier sont qu'il utilise une syntaxe plus large et permet une meilleure portabilité, il utilise `ON DELETE CASCADE` qui permet une meilleure intégrité des données et enfin on voit mieux quelle colonne fait quoi grâce aux `PRIMARY KEY` et des clés étrangères présent dans la création de la colonne.

Les avantages du deuxième sont qu'il utilise `serial` qui permet l'auto incrémentation à chaque ajout dans la colonne, il utilise `IF NOT EXIST`S qui empêche la création de doublon, les contraintes sont nommées et permet de les modifier ou de les supprimer plus facilement. De plus, il utilise `bigint` qui permet d'avoir plus d'identifiant comparé à `INTEGER`. Enfin, il sépare les définitions de table aux contraintes qui permettent une meilleure visibilité du script.

TRANSITION :

Nous venons de voir un moyen de modéliser d'une autre manière grâce à l'AGL, nous avons commenté des différences visuelles que l'AGL pouvait avoir avec le modèle vu en classe ainsi que les différences que l'AGL pouvait avoir lors de la génération de script comparé au script de la partie 1.

Ainsi, nous verrons comment remplir nos tables à l'aide d'un script, de fichier plat et de tables temporaires.

PARTIE 3
PEUPLEMENT DES TABLES
Premier jeux de données

1. Script du peuplement de la base de données :

```
CREATE TABLE region (
     region_code SERIAL PRIMARY KEY,
     name VARCHAR NOT NULL
     );

CREATE TABLE sub_region (
     sub_region_code SERIAL PRIMARY KEY,
     name VARCHAR NOT NULL,
     region_code INTEGER NOT NULL REFERENCES
     region(region_code) ON DELETE CASCADE
);

CREATE TABLE country (
     country_code SERIAL PRIMARY KEY,
     name VARCHAR NOT NULL,
     iso2 VARCHAR,
     iso3 VARCHAR NOT NULL,
     sub_region_code INTEGER NOT NULL REFERENCES
     sub_region(sub_region_code) ON DELETE CASCADE
);

CREATE TABLE disaster (
     disaster_code SERIAL PRIMARY KEY,
     disaster VARCHAR NOT NULL
);

CREATE TABLE climate_disaster (
     country_code INTEGER NOT NULL REFERENCES
     country(country_code) ON DELETE CASCADE,
     disaster_code INTEGER NOT NULL REFERENCES
     disaster(disaster_code) ON DELETE CASCADE,
     year INTEGER NOT NULL,
     number INTEGER NOT NULL,
     PRIMARY KEY (country_code, disaster_code, year)
);
```

```sql
CREATE TEMP TABLE temp_data (
     country VARCHAR NOT NULL,
     iso2 VARCHAR,
     iso3 VARCHAR NOT NULL,
     region_code INTEGER,
     region VARCHAR NOT NULL,
     sub_region_code INTEGER,
     sub_region VARCHAR NOT NULL,
     disaster VARCHAR NOT NULL,
     year INTEGER NOT NULL,
     number INTEGER NOT NULL
);
```

```sql
\copy temp_data(country, iso2, iso3, region_code, region,
sub_region_code, sub_region, disaster, year, number)
FROM 'Documents/sae_bdd/donnees.csv' CSV HEADER;
```

```sql
INSERT INTO region (name)
SELECT DISTINCT region
FROM temp_data
WHERE region IS NOT NULL;
INSERT INTO sub_region (name, region_code)
SELECT DISTINCT sub_region, region.region_code
FROM temp_data
JOIN region ON temp_data.region = region.name
WHERE sub_region IS NOT NULL;

INSERT INTO country (name, iso2, iso3, sub_region_code)
SELECT DISTINCT temp_data.country, temp_data.iso2,
temp_data.iso3, sub_region.sub_region_code
FROM temp_data JOIN sub_region ON temp_data.sub_region =
sub_region.name
WHERE temp_data.country IS NOT NULL;

INSERT INTO disaster (disaster)
SELECT DISTINCT disaster
FROM temp_data
WHERE disaster IS NOT NULL;
```

```sql
INSERT INTO climate_disaster (country_code,
disaster_code,year, number)
SELECT country.country_code, disaster.disaster_code,
temp_data.year, temp_data.number
FROM temp_data JOIN country ON temp_data.country =
country.name
JOIN disaster ON temp_data.disaster = disaster.disaster
WHERE temp_data.year IS NOT NULL AND temp_data.number IS
NOT NULL;

DROP TABLE temp_data;
```

<u>Résultat obtenu :</u>

Image de jointure de toutes les tables de sortes à avoir le tableau csv mais sur le SGBD

Le tableau est trop grand donc il est séparé en deux parties :

Gauche :

country	iso2	iso3	region_code	region	sub_region_code	sub_region	disaster	year	num
Costa Rica	CR	CRI	2	Americas	14	Latin America and the Caribbean	Drought	2018	1
Thailand	TH	THA	4	Asia	3	South-eastern Asia	Drought	2008	1
Brazil	BR	BRA	2	Americas	14	Latin America and the Caribbean	Drought	1985	1
Thailand	TH	THA	4	Asia	3	South-eastern Asia	Drought	2012	1
China, P.R.: Mainland	CN	CHN	4	Asia	15	Eastern Asia	Drought	1991	1
Sri Lanka	LK	LKA	4	Asia	8	Southern Asia	Drought	2019	1
Guatemala	GT	GTM	2	Americas	14	Latin America and the Caribbean	Drought	2001	1
Burkina Faso	BF	BFA	3	Africa	2	Sub-Saharan Africa	Drought	1990	1
Mozambique, Rep. of	MZ	MOZ	3	Africa	2	Sub-Saharan Africa	Drought	2021	1
Guyana	GY	GUY	2	Americas	14	Latin America and the Caribbean	Drought	1997	1
Burkina Faso	BF	BFA	3	Africa	2	Sub-Saharan Africa	Drought	2014	1
Cabo Verde	CV	CPV	3	Africa	2	Sub-Saharan Africa	Drought	2022	1
South Africa	ZA	ZAF	3	Africa	2	Sub-Saharan Africa	Drought	1995	1
Vietnam	VN	VNM	4	Asia	3	South-eastern Asia	Drought	1997	1
Tajikistan, Rep. of	TJ	TJK	4	Asia	17	Central Asia	Drought	2008	1
Madagascar, Rep. of	MG	MDG	3	Africa	2	Sub-Saharan Africa	Drought	1981	1
Indonesia	ID	IDN	4	Asia	3	South-eastern Asia	Drought	1997	1
Lao People's Dem. Rep.	LA	LAO	4	Asia	3	South-eastern Asia	Drought	1991	

Droite:

ub_region_code	sub_region	disaster	year	number
14	Latin America and the Caribbean	Drought	2018	1
3	South-eastern Asia	Drought	2008	1
14	Latin America and the Caribbean	Drought	1985	1
3	South-eastern Asia	Drought	2012	1
15	Eastern Asia	Drought	1991	1
8	Southern Asia	Drought	2019	1
14	Latin America and the Caribbean	Drought	2001	1
2	Sub-Saharan Africa	Drought	1990	1
2	Sub-Saharan Africa	Drought	2021	1
14	Latin America and the Caribbean	Drought	1997	1
2	Sub-Saharan Africa	Drought	2014	1
2	Sub-Saharan Africa	Drought	2022	1
2	Sub-Saharan Africa	Drought	1995	1
3	South-eastern Asia	Drought	1997	1
17	Central Asia	Drought	2008	1
2	Sub-Saharan Africa	Drought	1981	1
3	South-eastern Asia	Drought	1997	1
3	South-eastern Asia	Drought	1991	1
2	Sub-Saharan Africa	Drought	2005	1
2	Sub-Saharan Africa	Drought	2011	1
2	Sub-Saharan Africa	Drought	1984	1
5	Eastern Europe	Drought	2003	1
5	Eastern Europe	Drought	2000	1
2	Sub-Saharan Africa	Drought	2014	1
2	Sub-Saharan Africa	Drought	2013	1
9	Australia and New Zealand	Drought	1981	1
3	South-eastern Asia	Drought	2019	1
13	Micronesia	Drought	2013	1
3	South-eastern Asia	Drought	2005	1
2	Sub-Saharan Africa	Drought	2014	1
14	Latin America and the Caribbean	Drought	1988	1
7	Northern Africa	Drought	1990	1
2	Sub-Saharan Africa	Drought	2015	1
2	Sub-Saharan Africa	Drought	2022	1
2	Sub-Saharan Africa	Drought	1990	1
11	Melanesia	Drought	1983	1
2	Sub-Saharan Africa	Drought	1989	1

Nombre de lignes du tableau qui correspond bien aux lignes du fichier csv i.e 6449-1 sans compter la ligne de la colonne :

Nepal	NP	NPL	4	Asia	8	Southern Asia	Wildfire	2016
1								
Greece	GR	GRC	5	Europe	1	Southern Europe	Wildfire	1983
1								
South Africa	ZA	ZAF	3	Africa	2	Sub-Saharan Africa	Wildfire	2007
1								
Australia	AU	AUS	1	Oceania	9	Australia and New Zealand	Wildfire	2015
3								
Australia	AU	AUS	1	Oceania	9	Australia and New Zealand	Wildfire	2013
2								

(6448 rows)

Toutes les tables finales :

```
ordidellm=> \d
                    List of relations
 Schema |                Name                |   Type   |   Owner
--------+------------------------------------+----------+----------
 public | climate_disaster                   | table    | ordidellm
 public | country                            | table    | ordidellm
 public | country_country_code_seq           | sequence | ordidellm
 public | disaster                           | table    | ordidellm
 public | disaster_disaster_code_seq         | sequence | ordidellm
 public | region                             | table    | ordidellm
 public | region_region_code_seq             | sequence | ordidellm
 public | sub_region                         | table    | ordidellm
 public | sub_region_sub_region_code_seq     | sequence | ordidellm
(9 rows)
```

2. Description commenté des différentes étapes du script de peuplement

- ➔ On installe le fichier csv (plat) du sujet, on le désarchive et on l'ouvre avec Excel par exemple.

- ➔ On utilise un éditeur de code (visual studio code dans ce cas) et un SGBDR qui va nous permettre d'exécuter le code

- ➔ On analyse d'abord le fichier csv
  - ◆ on regarde les colonnes présentes
  - ◆ voir si les informations sont utilisables en cas de doublon par exemple

- ➔ On met d'abord le script de création de table, repris de la partie 1:
  - ◆ Il y a eu un problème au niveau du `ISO2` car il y a une ligne qui n'a pas de valeur donc il fallait supprimer la contrainte d'avoir 2 valeurs pour que cela fonctionne.
  - ◆ Le type `SERIAL` a aussi été mis en place pour les clés primaires dans le cas où elle n'était pas présente dans le fichier plat

- ➔ On crée une table temporaire, qu'on va appeler `temp_data` qui permet de recueillir les données du fichier plat où on fait bien attention de mettre tous les noms de colonnes ainsi que leurs types.

- ➔ On utilise ensuite la commande `\copy` en indiquant:
  - ◆ le chemin absolu vers le fichier csv
  - ◆ le délimiteur: ','
  - ◆ le type d'en-tête

- ➔ On insère les données directement dans les tables en utilisant `INSERT INTO` et une requête `SELECT` pour chaque table:
  - ◆ `INSERT INTO` permet de mettre des informations dans tables en précisant la table, la/les colonnes et les valeurs dans l'ordre des colonnes.
  - ◆ `SELECT` permet d'extraire les informations de la table temporaire `temp_data`. Lorsque nécessaire, on fait des jointures pour entre les données de `temp_data` (qui reprend les données du fichier plat) et les clés des tables principales pour avoir les bonnes associations et pour qu'elles soient exploitables dans la base.

- `WHERE … IS NOT NULL` permet d'éviter d'entrer des valeurs nulles ou manquantes dans les tables.

➔ On supprime finalement la table temporaire `DROP TABLE` qui permet à la fois de libérer de l'espace et elle n'a plus d'utilité après.

➔ Enfin, on vérifie si on a bien eu le peuplement voulu. (résultat dans les captures d'écrans précédentes)

- `\d` pour avoir le nom des tables
-
```
SELECT
        country.name AS country,
        country.iso2,
        country.iso3,
        region.region_code,
        region.name AS region,
        sub_region.sub_region_code,
        sub_region.name AS sub_region,
        disaster.disaster,
        climate_disaster.year,
        climate_disaster.number
FROM climate_disaster
JOIN country ON climate_disaster.country_code =
country.country_code
JOIN sub_region ON country.sub_region_code =
sub_region.sub_region_code
JOIN region ON sub_region.region_code =
region.region_code
JOIN disaster ON climate_disaster.disaster_code
= disaster.disaster_code;
```

CONCLUSION :

Pour conclure, nous avons commencé par faire un script de création de tables principalement fait à l'aide des connaissances du cours.
On a ensuite vu un nouveau moyen de modéliser les bases de données où on a d'abord comparé visuellement cette modélisation et celle du cours.
On a ensuite généré un script depuis l'AGL pour regarder les différences aux niveaux des scripts écrits à la main et celui de l'AGL. On a vu qu'il y avait quelques différences au niveau de la forme principalement comme les clés référencés en dehors de la création de table.
Enfin, on a donc "peupler" les tables qui étaient à l'origine vide à l'aide d'un nouveau script qu'on devait concevoir avec un fichier csv fourni par le sujet. On a donc repris le script de création de table de la partie 1 puis l'améliorer avec celle de la partie 2, on a créé une table temporaire qui recueille les informations du fichier plat, on a ajouté des insertions pour remplir les tables et enfin on a supprimé cette table temporaire.
Tout ce travail nous a permis de voir qu'on pouvait modéliser autrement que par la modélisation du cours grâce à un AGL (atelier génie logiciel) et qu'on pouvait "peupler" des tables grâce à une table temporaire et des insertions.